

# ARCA: Adapter-Residual Credit Assignment

## When Token Signals Degenerate

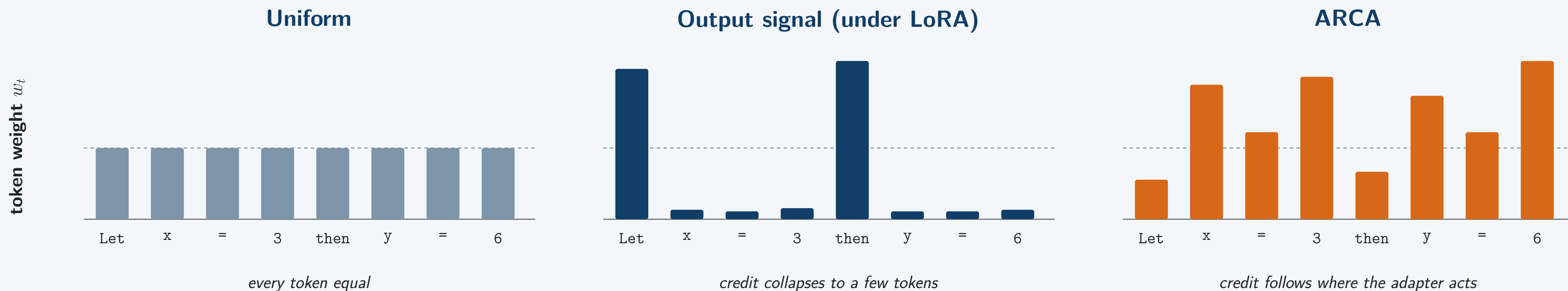
Rodney Lafuente-Mercado · Scale AI · rodney.lafuente@scale.com

DEMO 2026 · ICML Workshop on Decision-Making from Offline Datasets to Online Adaptation · [github.com/rodlaft/tokenweighting](https://github.com/rodlaft/tokenweighting)

**TL;DR.** Practical LLM-RL runs on **LoRA**, but standard token-credit signals (surprisal, entropy reduction, policy divergence) **degenerate** under low-rank adaptation. They collapse either to uniform broadcast or to spurious sparsity. **ARCA** instead reads salience straight from the adapter’s own hidden-state residual  $\|h_t^{\text{adapted}} - h_t^{\text{base}}\|_2$ , with no critic, no reward model, and no trees. It keeps credit in a useful middle regime and stays competitive at matched parameters.

### How ARCA assigns token credit

Each token’s pull on the gradient in  $g = A \Sigma_t w_t \nabla \log \pi_\theta(y_t)$  is its weight  $w_t$  (bar height below). ARCA sets  $w_t = \|h_t^{\text{adapted}} - h_t^{\text{base}}\|_2$ , so the tokens the adapter changes most get the most credit. The dashed line marks the uniform weight.



### 1. The Problem

- RL drives LLM post-training, but **credit assignment** is hard: trajectories are long and the reward is a single sparse outcome scalar.
- Intrinsic token weighting reads per-token salience off the model’s own output distribution (surprisal, entropy, divergence).
- Yet **most real LLM-RL pipelines use LoRA** for memory and compute.
- These threads are studied apart. We show their **interaction** is the real issue.

### 2. Failure Mode: Signal Degeneration under LoRA

LoRA confines the policy  $\pi_\theta$  to a low-rank neighborhood of the reference  $\pi_{\text{ref}}$  (an implicit KL bound), so the per-token output differences these signals measure are

$$-\log \pi_\theta(y_t) = -\log \pi_{\text{ref}}(y_t) + O(\|\Delta h_t\|),$$

**dominated by the base model**, not the adapter. After within-trajectory normalization the weights degenerate:

- Uniform broadcast:** Gini  $G(w) \approx 0$  when the  $\varepsilon$  floor dominates.
- Spurious sparsity:**  $G(w) \approx 1$  with a tiny effective support  $\text{EffN}/T$ .

Either way the weights stop tracking where the adapter acts. Read it off directly with **Gini**  $G(w)$  and the **effective-token ratio**  $\text{EffN}(w)/T = 1/(T \Sigma_t w_t^2)$ .

### 3. ARCA

Measure salience where LoRA actually acts, in the adapter’s own contribution to the forward pass:

$$\alpha_t^{\text{ARCA}} = \|h_t^{\text{adapted}} - h_t^{\text{base}}\|_2$$

then normalize within each trajectory to weights  $w_t^{\text{ARCA}}$ .

- One extra no-grad forward pass with `model.disable_adapter()`, the same call already used for the KL term or divergence weighting. No new infrastructure.
- Weights are **detached**: no second-order terms, cost close to a critic-free policy gradient.
- No value head, no learned process reward model, no search trees.

### 4. Why ARCA Avoids Collapse

- Even when  $\|B_\ell A_\ell\|$  is small, the adapter input activations  $x_{\ell,t}$  stay **non-uniform across positions** (attention, layer norm, content vs filler), so  $\|\Delta h_\ell\|$  keeps real position-to-position variation.
- It reads as cheap **gradient-informed** credit: a large residual marks positions where a policy-gradient update has the most effect.
- With a fixed  $\varepsilon$  floor, any score whose magnitude vanishes eventually normalizes to uniform. ARCA’s residual stays measurable, so its weights remain tied to adapter activity instead of to floor numerics.

### 5. Experimental Setup

- Qwen3-1.7B-Base, **MATH**, **GRPO**, binary exact-match reward, LoRA on attn and MLP.
- $K = 4$  completions per prompt, 100 update steps, seed 1337; eval greedy accuracy and pass@4 on 200 held-out.
- 7 runs:** uniform, surprisal, entropy reduction, policy divergence at  $r=64$ ; ARCA at  $r \in \{4, 16, 64\}$ .
- AdamW at  $\text{lr } 5 \times 10^{-6}$ , 10-step warmup then cosine decay, batch size 2 with 4-step gradient accumulation, prompt and generation length capped at 512, temperature 1.0, top- $p$  0.95.
- Same rollout groups, optimizer, schedule, and decoding across methods, so token redistribution is the only algorithmic difference. Each run fits on one H100-class GPU in a few hours.

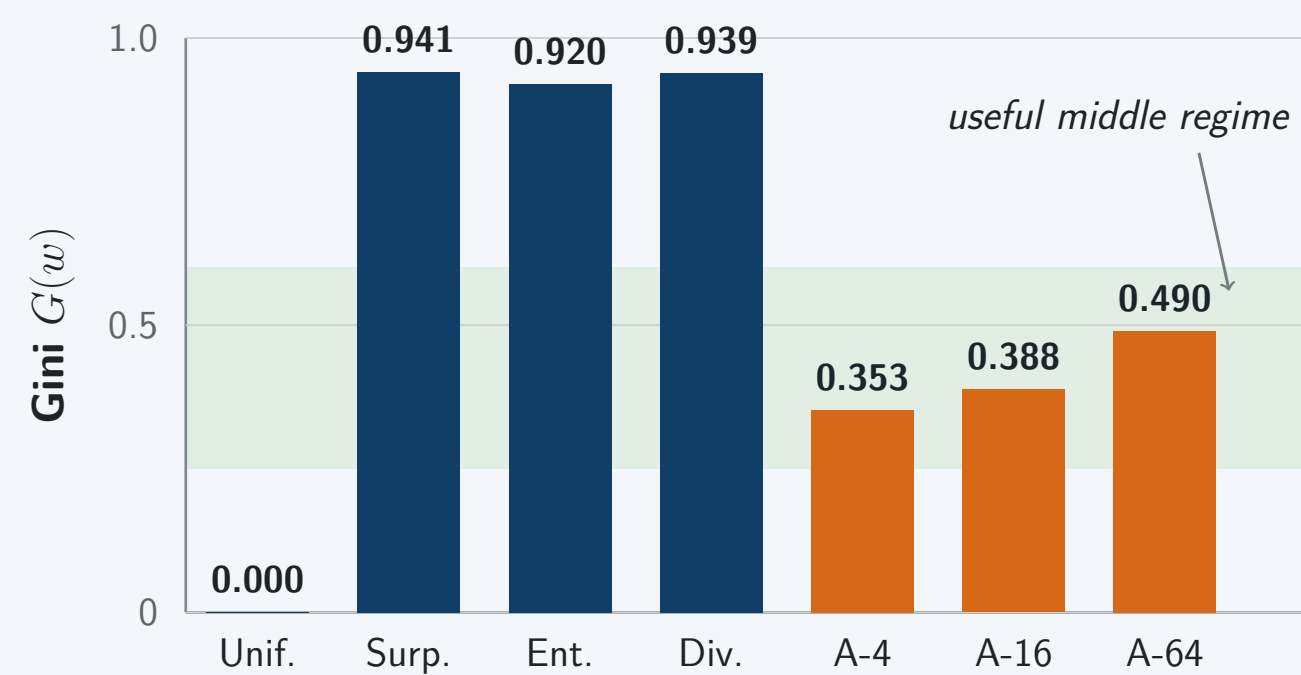
### 6. Held-out MATH Performance

| Method            | $r$ | Greedy       | Pass@4       | Tr. rew. |
|-------------------|-----|--------------|--------------|----------|
| Uniform           | 64  | <b>0.620</b> | 0.675        | 0.362    |
| Surprisal         | 64  | 0.520        | 0.650        | 0.326    |
| Entropy reduction | 64  | 0.600        | <b>0.700</b> | 0.367    |
| Policy divergence | 64  | 0.565        | 0.670        | 0.351    |
| ARCA              | 4   | 0.525        | 0.640        | 0.327    |
| ARCA              | 16  | 0.510        | 0.655        | 0.336    |
| ARCA              | 64  | 0.590        | 0.680        | 0.353    |

At matched rank 64, ARCA is competitive: 3 points under uniform on greedy and 0.5 point over it on pass@4. Lower-rank ARCA trails the rank-64 baselines, which points to a capacity effect rather than an extra-parameter effect. No single weighting wins outright here: uniform takes greedy and entropy reduction takes pass@4, so read this table as a compact validation, not a benchmark sweep.

### 7. The Central Result: Credit Concentration

| Method            | $r$ | $G(w)$ | $\text{EffN}/T$ |
|-------------------|-----|--------|-----------------|
| Uniform           | 64  | 0.000  | 1.000           |
| Surprisal         | 64  | 0.941  | 0.052           |
| Entropy reduction | 64  | 0.920  | 0.073           |
| Policy divergence | 64  | 0.939  | 0.055           |
| ARCA              | 4   | 0.353  | 0.467           |
| ARCA              | 16  | 0.388  | 0.460           |
| ARCA              | 64  | 0.490  | 0.334           |



The baselines sit at the extremes ( $G \approx 0$  or  $\approx 0.94$ ). ARCA lands in the predicted middle: non-uniform, but not collapsed onto a handful of tokens. Output signals ask where the model is uncertain; ARCA asks where the adapter acts.

### 8. Contributions

- We formalize a PEFT-specific failure mode: under LoRA, token salience collapses to uniform broadcast or spurious sparsity.
- We introduce **ARCA**, an adapter-residual credit rule whose weights stay non-degenerate when the adapter varies by position.
- We validate it with concentration diagnostics and a matched MATH/Qwen3-1.7B sweep.

### 9. Takeaways

- Token-level credit assignment and PEFT are **coupled**, not orthogonal.
- Under LoRA, output-distribution signals degenerate, so a null result for “elaborate weighting vs uniform” can be a LoRA artifact.
- ARCA** gives an adaptation-aware, lightweight, non-degenerate credit signal that stays competitive at matched parameters.